

Deep Learning Project

Tom Garcia Thomas d'Arc

Avril 2026

1 Introduction : choix du sujet, références

J'ai choisi de travailler sur la comparaison de deux légers modèles de classification d'images : *resnet18*, un CNN, et *vit-tiny-patch16-224*, un Vision Transformer. Ils sont tous les deux **pré-entraînés** sur ImageNet1K une base de données largement utilisée. On les dénommera respectivement CNN et ViT dans la suite du rapport.

Contrairement à la ressource communiquée dans le sujet du projet, je ne me suis pas basé sur le papier Lu and al. (2022) qui présente en fait une architecture de Vision Transformer Hybride assez poussée ce qui aurait probablement nuit à une comparaison stricte et pure entre les deux types de modèles. On se basera plutôt sur les articles originaux de chacun des modèles :

- ResNet : Krizhevsky and al. (2012). ImageNet classification with deep convolutional neural networks.¹
- ViT : Dosovitskiy and al. (2020). An image is worth 16x16 words : Transformers for image recognition at scale.

Après une étude et une compréhension (relative) du fonctionnement d'un CNN et d'un ViT, on peut faire les hypothèses suivantes sur les différences de performances entre les deux modèles :

- **(H1)** Le CNN permet de capturer beaucoup de caractéristiques locales dans les images grâce à ses couches de convolution => il devrait être plus performant sur des motifs répétitifs et textures localement similaires.
- **(H2)** Le ViT permet de capturer des relations globales dans les images grâce à son mécanisme d'attention => il devrait être plus performant sur des motifs qui nécessitent une compréhension globale de l'image mais aussi donc être plus robuste aux variations de textures.

On peut donc construire des datasets artificiels pour tester ces hypothèses.

1. Le modèle resnet utilise une architecture qui introduit en plus des CNN originaux des connexions résiduelles (skip connections)

Modèle	NB de paramètres	Size (MB)	Data de pré-entraînement	Accuracy@5
ResNet18	11,6M	45	ImageNet1K	85%
ViT-tiny-patch16-224	5,7M	23	ImageNet1K	Non-fournie

TABLE 1 – Caractéristiques des modèles utilisés, ViT est plus léger mais les deux modèles ont même ordre de grandeur de paramètres et de taille.

2 Datasets et Méthodologie

2.1 Choix du dataset

Le choix du dataset est donc crucial pour tester les hypothèses formulées précédemment, il faut un dataset suffisamment simple pour un entraînement rapide et diversifié pour tester les différentes capacités des modèles. Pour cela j'ai exploré plusieurs datasets sur HuggingFace et finalement opté pour le dataset *EUROSTAT* qui contient des images satellite de différentes classes de terrains (forêt, eau, bâtiments, etc.) labellées à la main. Le dataset est standardisé et propre [1] notamment déjà un split train/test et il représente équitablement les classes. De plus, il contient des motifs locaux (textures de forêts, d'eau) et des motifs globaux (disposition des bâtiments, formes des cours d'eau) ce qui paraît idéal pour tester les hypothèses **(H1)** et **(H2)**.



FIGURE 1 – *EUROSTAT* dataset : 5 images par classe.

Dataset	Nb d'images	Résolution(pixels)	Nb de classes	Images par classe
EUROSTAT RGB	27k	64x64	10	2000-3000

TABLE 2 – Caractéristiques du dataset *EUROSTAT*.

2.2 Méthodologie de validation des hypothèses

Ce dataset sera donc notre référence pour les expériences notamment l'entraînement, cependant pour tester plus précisément les hypothèses on peut aussi construire des datasets artificiels à partir de celui-ci. En effet, *EUROSTAT* contient des images de faible dimension (64x64 pixels) et reste tout de même texture-orienté ce qui risque d'avantager CNN lors du fine-tuning.

On va ainsi construire 3 types supplémentaires de **datasets de validation** à partir d'*EUROSTAT* :

- **Dataset de type 1** : On va appliquer un carré noir centré sur les images pour briser la localité : voir si l'information à distance est partagée 2.
- **Dataset de type 2** : On va appliquer un flou gaussien pour atténuer les textures locales et favoriser les motifs globaux 3.
- **Dataset de type 3** : On va appliquer un mask aléatoire sur un pourcentage des pixels pour introduire du bruit et tester la robustesse des modèles 4.

On va donc premièrement fine-tuner les deux modèles sur le dataset *EUROSTAT* standard et en se basant sur les métriques (K. He et al., 2015 [2]) et (Dosovitskiy et al., 2020 [3]) les évaluer sur les 3 datasets de validation avec comme métrique l'accuracy@1 (accuracy sur la première prédiction) mais aussi l'accuracy@2 qui est plus qualitative et permet de réduire l'overfitting sur les classes difficiles et évaluer "l'intelligence" des modèles. On évaluera aussi l'accuracy par classe sur un dataset où les deux modèles semblent performer aussi bien, mais font suffisamment d'erreur pour espérer noter une différence inter-classe et donc déduire des propriétés respectives différentes.

3 Entraînement des modèles

3.1 Pré-traitement des données

L'entraînement des modèles passe d'abord par une standardisation des images. En effet les modèles pré-entraînés sur ImageNet1K ont été entraînés sur des images de taille (224x224) pixels, il est donc nécessaire de redimensionner les images (64x64 pixels) à cette taille pour pouvoir les utiliser. Inconvénient, car réduit l'information par images et induit un flou obligatoire (alors qu'on conseille le fine-tune des ViT sur données de plus grandes dimensions encore).

De plus, les modèles ont été entraînés sur des images normalisées avec une moyenne et un écart-type spécifiques pour les 3 canaux de couleur (R, G, B), pour le CNN : $mean = [0.485, 0.456, 0.406]$, $std = [0.229, 0.224, 0.225]$ et pour le ViT : $mean = [0.5, 0.5, 0.5]$, $std = [0.5, 0.5, 0.5]$. Cela passe par la création d'une classe *AugmentedDataset* qui applique ces transformations² aux images du dataset.

3.2 Fine-tuning des modèles

Ensuite, on peut fine-tuner les modèles. Le choix que j'ai fait est de ne pas geler les poids des modèles pré-entraînés, afin de permettre une adaptation plus complète aux nouvelles classes du dataset *EUROSTAT*. Ainsi, je remplace uniquement la tête de classification par une nouvelle couche linéaire adaptée au nombre de classes (10 classes).

Pour l'entraînement, j'ai utilisé une cross-entropy loss (standard dans les tâches de classification multi-classe), un optimiseur Adam avec un learning rate $lr = 10^{-5}$ sur 10 epochs et early stopping lors de la baisse de la validation loss.

L'optimize Adam n'est lui pas forcément standard pour le fine-tuning, on peut penser à un SGD avec momentum (Dosovitskiy et al., 2020 [3]) avec grid search sur les learning rates, mais j'ai choisi Adam pour sa simplicité et rapidité de convergence (on peut aussi utiliser AdamW pour ajouter un weight decay qui peut aider à la généralisation, ici j'utilise uniquement l'early stopping, ce qui n'est pas forcément optimal contre l'overfitting sur petit dataset...).

3.3 Hardware

Pour les caractéristiques hardware, j'ai effectué l'entraînement sur un GPU T4 Google Colab, un batch size de 32 (à savoir que les images sont donc normalisées en (224x224) et 3 canaux, d'où le faible batch size) et num workers de 2 (privilegiés sur Google Colab).

Modèle	Temps d'entraînement	Loss finale (normalisé sur nb d'images)	Early Stop
CNN	9min	0.0632 (proba gold label : 0.9387 en moyenne)	8 epochs
ViT	4min	0.0804 (proba gold label : 0.9227 en moyenne)	3 epochs

TABLE 3 – Données d'entraînement

2. Ces transformations doivent aussi être utilisées avant l'inférence de validation pour les Datasets artificiels 1,2,3.

4 Présentation des résultats, discussion

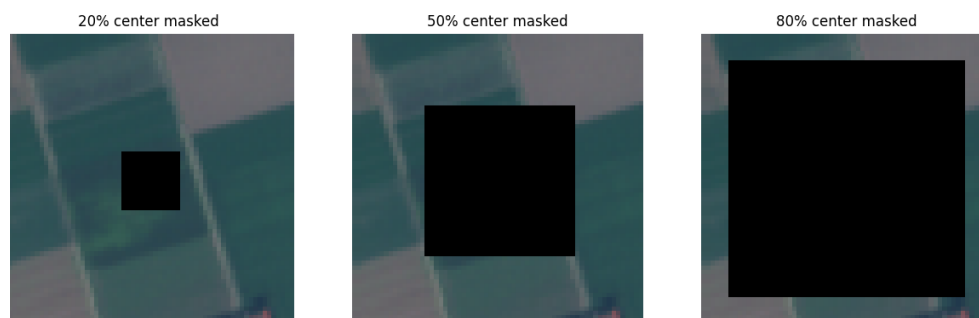


FIGURE 2 – Dataset 1 : 20, 50, 80 center-mask.



FIGURE 3 – Dataset 2 : 0.5, 1, 3 gaussian-blur.

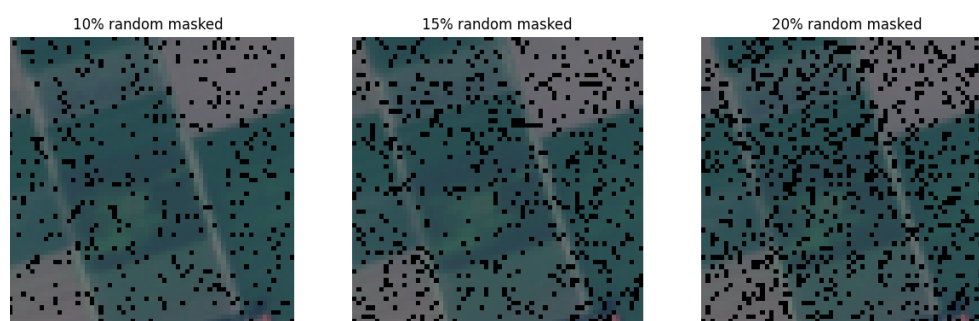


FIGURE 4 – Dataset 3 : 10, 15, 20 random-mask.

On performe donc l'accuracy@1 et l'accuracy@2 sur tous nos datasets. Pour clarifier le propos et les notations, on affiche une image du dataset original et les différentes transformations 1,2,3 ci-dessus (transformations antérieures aux normalisations, qui elles, ne donne pas d'images visualisables).

Le coût computationnel étant élevé (création de plusieurs dataset et transformation sur images), on les compute également sur Google Colab.

dataset	cnn	vit
no effect	0.981	0.975
20_center_masked	0.968	0.886
50_center_masked	0.641	0.505
80_center_masked	0.113	0.115
10_random_masked	0.113	0.286
15_random_masked	0.113	0.216
20_random_masked	0.113	0.199
0.5_gaussian_blur	0.972	0.972
1_gaussian_blur	0.742	0.808
3_gaussian_blur	0.160	0.345

TABLE 4 – Accuracy@1

dataset	cnn	vit
no effect	0.998	0.996
20_center_masked	0.994	0.968
50_center_masked	0.769	0.883
80_center_masked	0.299	0.506
10_random_masked	0.224	0.462
15_random_masked	0.236	0.372
20_random_masked	0.307	0.341
0.5_gaussian_blur	0.998	0.996
1_gaussian_blur	0.908	0.925
3_gaussian_blur	0.256	0.512

TABLE 5 – Accuracy@2

4.1 Première analyse des résultats

Premièrement, les accuracy@1 pour CNN sur le dataset de validation originel (sans effet) est de 98.1%, ce qui est cohérent avec le papier *EUROSAT* qui atteint 98.57% sur un *ResNet50*. On se convainc donc aussi que le dataset est "facile" même pour ViT. On obtient des informations à partir de l'acc@1 :

- 1. CNN est légèrement plus performant sur le dataset originel.
- 2. Plus le *center_mask* est élevé, plus le ViT tend à performer aussi bien que CNN, même si leurs performances ne sont pas claires.
- 3. *random_mask* détruit la structure local pour CNN (0.11 est proche d'une décision aléatoire).
- 4. *gaussian_blur* dégrade significativement la performance de CNN en comparaison à ViT (0.16 contre 0.345 pour *radius* = 3).

De l'acc@2 on peut noter quantitativement que ViT est plus robuste dans ces choix pour tous les datasets transformés. Ce n'est pas forcément surprenant car on a volontairement modifié ces datasets pour détruire la texture et la structure locales. On peut tout de même noter que pour *50_center_masked*, l'acc@1, qui est de 0.641 et 0.505 respectivement pour CNN et ViT, passe à 0.769 et 0.883.

Une amélioration énorme pour ViT, mais des performances similaires. Cela peut nous amener à évaluer les accuracy par classes dans ce contexte précis pour essayer de comprendre qualitativement leur différences.

4.2 Étude des métriques dans le cas particulier d'un dataset de type 1 : 50_center_masked

On performe donc les acc@1 et acc@2 dans ce cas particulier, mais pour chaque classe, c'est-à-dire pour chaque type de terrain et de formes différentes on essaye de mesurer les performances des modèles. Pour cela, il faut avoir en tête les types de terrains³.

class	cnn	vit	class	cnn	vit
AnnualCrop	0.960	1.000	AnnualCrop	0.997	1.000
Forest	0.887	0.363	Forest	0.985	0.982
HerbaceousVegetation	0.260	0.417	HerbaceousVegetation	0.382	0.942
Highway	0.254	0.264	Highway	0.419	0.454
Industrial	0.982	0.661	Industrial	1.000	0.870
Pasture	0.076	0.152	Pasture	0.131	0.773
PermanentCrop	0.721	0.439	PermanentCrop	0.870	0.942
Residential	0.466	0.928	Residential	0.830	0.964
River	0.575	0.406	River	0.836	0.832
SeaLake	0.993	0.304	SeaLake	1.000	0.964

TABLE 6 – Accuracy@1 par classe, cas : 50_center_mask

TABLE 7 – Accuracy@2 par classe, cas : 50_center_mask

Ce qu'on peut tirer de ces résultats :

- 5. Pour des terrains complètement uniformes (forêts, lacs), CNN performe bien mieux que ViT pour l'acc@1 (0.887 vs 0.363 et 0.993 vs 0.304). Ce défaut est gommé à l'acc@2 (0.985 vs 0.982 et 1.000 vs 0.964). On en déduit que ViT confond les deux terrains uniformes du dataset plus souvent, il ne comprend pas la texture en elle-même mais l'uniformité de celle-ci (cela peut être aussi influencé par l'uniformité du maskage central qu'on ajoute).
- 6. Dans les classes difficiles qui se ressemblent (*HerbaceousVegetation*, *Pasture*, *PermanentCrop*), enlever un carré central impacte beaucoup plus CNN, qui ne peut que sommer des texture localement et donc ici aux bords.

Les observations effectuées ont tendance à aller dans le sens de nos hypothèses (H1) et (H2), même si des améliorations sont possibles pour les valider plus explicitement.

3. Voir Figure 1.

5 Amélioration de l'étude

Ici on discute de la validité de nos résultats et des améliorations possibles.

5.1 Choix du dataset

Ce dataset n'était pas forcément optimal pour différencier à lui seul les différences entre CNN et ViT. En effet, il présente des défauts majeurs : faible résolution, faible diversité d'images. D'où une méthode peut-être (trop) artificielle pour noter des divergences entre les modèles.

5.2 Entraînement

Une amélioration possible outre les choix d'hyperparamètres, de Cross-Validation, de choix d'optimiseurs qui chercheraient à augmenter la performance des modèles, un choix pertinent aurait été d'augmenter le dataset d'entraînement. Notamment avec ce qui est généralement fait : des changements de luminosité, des rotations et, pourquoi pas, en ajoutant les dataset de type 1,2,3 qu'on a créé. On aurait pu aussi rajouter du dropout qui n'est pas présent dans ces petits modèles. Bien sûr, le but étant d'accorder tous ce choix pour augmenter l'apprentissage sans overfitting.

5.3 Évaluation qualitative

Bien que l'acc@1 et l'acc@2 soit probablement des métriques quantitatives intéressantes, des rajouts visuels comme les kernels de convolutions pour CNN où la norme des attentions par patch dans certaines images pour ViT aurait pu nous éclairer d'avantage sur les mécanismes de selections de fréquences/couleurs et d'attention particulièrement dans nos cas dégénérés de type 1,2,3.

6 Structure du projet

```
deep_learning_project/  
|  
+-- main.pdf  
+-- python/  
|   +-- data.py  
|   +-- models.py  
|   +-- train.py  
|   +-- utils.py  
+-- notebooks/  
|   +-- dataset_overview.ipynb  
|   +-- evaluation.ipynb  
|   +-- models_training.ipynb  
+-- models/  
+-- images/
```



```
+-- logs/  
+-- README.md/  
+-- requirements.txt
```

Contenu des dossiers :

- `main.pdf` : Rapport du Projet
- `data.py` : contient le pre-processing des données.
- `models.py` : contient le chargement des modèles pre-entraînés, les fonctions d'accuracy.
- `train.py` : la fonction d'entraînement.
- `utils.py` : fonctions utiles.
- `dataset_overview.ipynb` : exploration du dataset EUROSTAT.
- `evaluation.ipynb` : fournit les logs d'accuracy.
- `models_training.ipynb` : entraîne les modèles.
- `models/` : stocke les modèles.

Code disponible sur Git-Hub.

Références

- [1] Patrick Helber and al. *EuroSAT : A Novel Dataset and Deep Learning Benchmark for Land Use and Land Cover Classification*, 2017
- [2] K. He et al. *Deep Residual Learning for Image Recognition*, 2015.
- [3] Dosovitskiy and al. *An image is worth 16x16 words : Transformers for image recognition at scale* 2020.